Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To obtain the best fit line over single feature scattered datapoints using Linear Regression**	
Experiment No: 02	**Date: 10/2/2026**	**Enrollment No: 92301733059**

Aim: To obtain the best fit line over single feature scattered datapoints using Linear Regression

IDE: Google Colab

Theory:

Linear regression is a method for determining the best linear relationship between two variables *X* and *Y*. If variables *X* and *Y* are uncorrelated, it is pointless embarking upon linear regression. However, if a reasonable degree of correlation exists between *X* and *Y* then linear regression may be a useful means to describe the relationship between the two variables. The usual approach is to use the *least-squares* method, which minimizes the squared difference between the actual data points and a straight line. Let $[x_i,y_i]$, i = 1,2,3,….,N be the N pairs of data values of the variables *X* and *Y*. The straight-line relating *X* and *Y* is $y = mx + c$, where m and c are the gradient and constant values (to be determined) defining the straight line. Thus, $y(x_i) - y_i$ is the difference between the line and data point i (see Fig. 1). Taking all the data points, we seek values of m and c that minimize the squared difference *SD*.

$$\sum_{1}^{N}\left[y(x_i)-y_i\right]^2$$

This is achieved by calculating the partial derivatives of *SD* with respect to m and c and finding the pair $[m,c]$ such that *SD* is at a minimum.

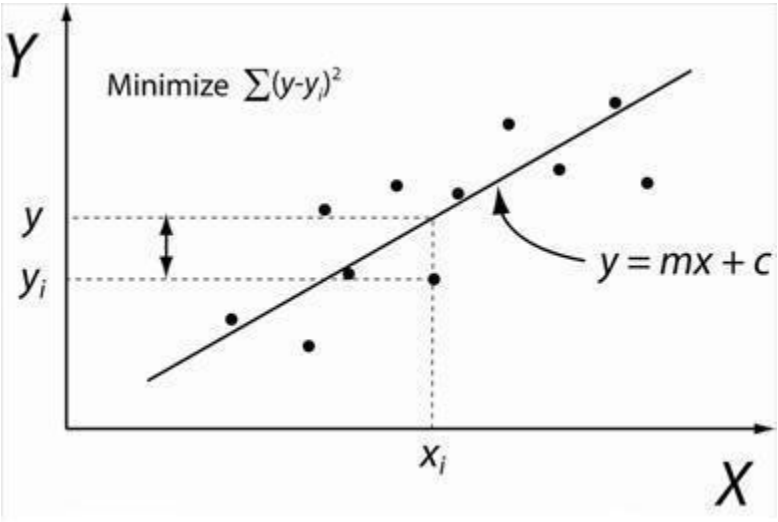


Figure 1: Illustration of Linear Regression. Linear least squares regression, the idea is to find the line y = mx + c that minimizes the mean squared difference between the line and the data points

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To obtain the best fit line over single feature scattered datapoints using Linear Regression**	
Experiment No: 02	**Date: 10/2/2026**	**Enrollment No: 92301733059**

Batch Gradient Descent:

Gradient Descent is an optimization algorithm used for minimizing the cost function in various machine learning algorithms. It is basically used for updating the parameters of the learning model. Batch gradient descent which processes all the training examples for each iteration of gradient descent. But if the number of training examples is large, then batch gradient descent is computationally very expensive.

Let m be the number of training examples. Let n be the number of features.

Algorithm for batch gradient descent :

Let $h_\theta(x)$ be the hypothesis for linear regression. Then, the cost function is given by:
Let Σ represents the sum of all training examples from i=1 to m.

$$J_{train}(\theta) = (1/2m)\ \Sigma(\ h_\theta(x^{(i)}) - y^{(i)})^2$$

Repeat {

$$\theta j = \theta j - (\text{learning rate}/m) * \Sigma(\ h_\theta(x^{(i)}) - y^{(i)})x_j^{(i)}$$

For every j =0 …n

}

Where $x_j^{(i)}$ Represents the j^{th} feature of the i^{th} training example. So if *m* is very large(e.g. 5 million training samples), then it takes hours or even days to converge to the global minimum.That's why for large datasets, it is not recommended to use batch gradient descent as it slows down the learning.

Pre Lab Exercise:

a. Explain the meaning of linear regression

b. Write three applications of linear regression

c. Write three advantages of linear regression

d. Write three limitations of linear regression

Methodology:

1. Load the basic libraries and packages
2. Load the dataset
3. Analyse the dataset
4. Pre-process the data
5. Visualize the Data

Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To obtain the best fit line over single feature scattered datapoints using Linear Regression	
Experiment No: 02	Date: 10/2/2026	Enrollment No: 92301733059

6. Separate the feature and prediction value columns
7. Write the Hypothesis Function
8. Write the Cost Function
9. Write the Gradient Descent optimization algorithm
10. Apply the training over the dataset to minimize the loss
11. Find the best fit line to the given dataset
12. Observe the cost function vs iterations learning curve

Program (Code):

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import PolynomialFeatures
from mpl_toolkits.mplot3d import Axes3D

class SimpleLinearRegression:
    def __init__(self):
        self.coefficient_ = None
        self.intercept_ = None
        self.r2score_ = None

    def fit(self, X, y):
        n = len(X)
        X_b = np.c_[np.ones((n,1)), X]  # Add bias
        # Calculate coefficients using Normal Equation
        self.coefficients_ = np.linalg.inv(X_b.T.dot(X_b)).dot(X_b.T).dot(y)
        self.intercept_ = self.coefficients_[0]
        # Predictions
        y_pred = X_b.dot(self.coefficients_)
        # R²
        self.r2score_ = 1 - (np.sum((y - y_pred)**2) / np.sum((y -
np.mean(y))**2))
        self.y_pred_ = y_pred

    def predict(self, X):
        X_b = np.c_[np.ones((len(X),1)), X]
        return X_b.dot(self.coefficients_)
X_simple = np.array([1,2,3,4,5,6,7,8,9,10]).reshape(-1,1)
y_simple = np.array([2,4,5,4,5,7,8,9,10,12])
```

Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To obtain the best fit line over single feature scattered datapoints using Linear Regression	
Experiment No: 02	Date: 10/2/2026	Enrollment No: 92301733059

```
slr = SimpleLinearRegression()
slr.fit(X_simple, y_simple)
print(f"Simple LR Coefficients: {slr.coefficients_}")
print(f"R² Score: {slr.r2score_:.2f}")

plt.scatter(X_simple, y_simple, color='blue', label='Data')
plt.plot(X_simple, slr.y_pred_, color='red', label='Regression Line')
plt.title("Simple Linear Regression")
plt.xlabel("X")
plt.ylabel("y")
plt.legend()
plt.show()

class MultipleLinearRegression:
    def __init__(self):
        self.coefficients_ = None
        self.intercept_ = None
        self.r2score_ = None

    def fit(self, X, y):
        n = X.shape[0]
        X_b = np.c_[np.ones((n,1)), X]  # Add bias
        self.coefficients_ = np.linalg.inv(X_b.T.dot(X_b)).dot(X_b.T).dot(y)
        self.intercept_ = self.coefficients_[0]
        y_pred = X_b.dot(self.coefficients_)
        self.r2score_ = 1 - (np.sum((y - y_pred)**2)/np.sum((y -
np.mean(y))**2))
        self.y_pred_ = y_pred

    def predict(self, X):
        X_b = np.c_[np.ones((X.shape[0],1)), X]
        return X_b.dot(self.coefficients_)

np.random.seed(0)
X1 = np.random.randint(1, 11, 15)
X2 = np.random.randint(1, 11, 15)
X_multi = np.column_stack((X1, X2))
y_multi = 1 + 2*X1 + 3*X2 + np.random.randn(15)*2
```

Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To obtain the best fit line over single feature scattered datapoints using Linear Regression	
Experiment No: 02	Date: 10/2/2026	Enrollment No: 92301733059

```
mlr = MultipleLinearRegression()
mlr.fit(X_multi, y_multi)
print(f"Multiple LR Coefficients: {mlr.coefficients_}")
print(f"R² Score: {mlr.r2score_:.2f}")


fig = plt.figure(figsize=(10,7))
ax = fig.add_subplot(111, projection='3d')

ax.scatter(X_multi[:,0], X_multi[:,1], y_multi, color='blue', label='Data')

x1_surf, x2_surf = np.meshgrid(
    np.linspace(X_multi[:,0].min(), X_multi[:,0].max(), 10),
    np.linspace(X_multi[:,1].min(), X_multi[:,1].max(), 10)
)


pred_surf = mlr.predict(np.c_[x1_surf.ravel(),
x2_surf.ravel()]).reshape(x1_surf.shape)


ax.plot_surface(x1_surf, x2_surf, pred_surf, color='red', alpha=0.5, rstride=1,
cstride=1)

ax.set_xlabel('X1')
ax.set_ylabel('X2')
ax.set_zlabel('y')
ax.set_title("Multiple Linear Regression with Regression Plane")
ax.legend()
plt.show()

class PolynomialRegression:
    def __init__(self, degree=2):
        self.degree = degree
        self.coefficients_ = None
        self.intercept_ = None
```

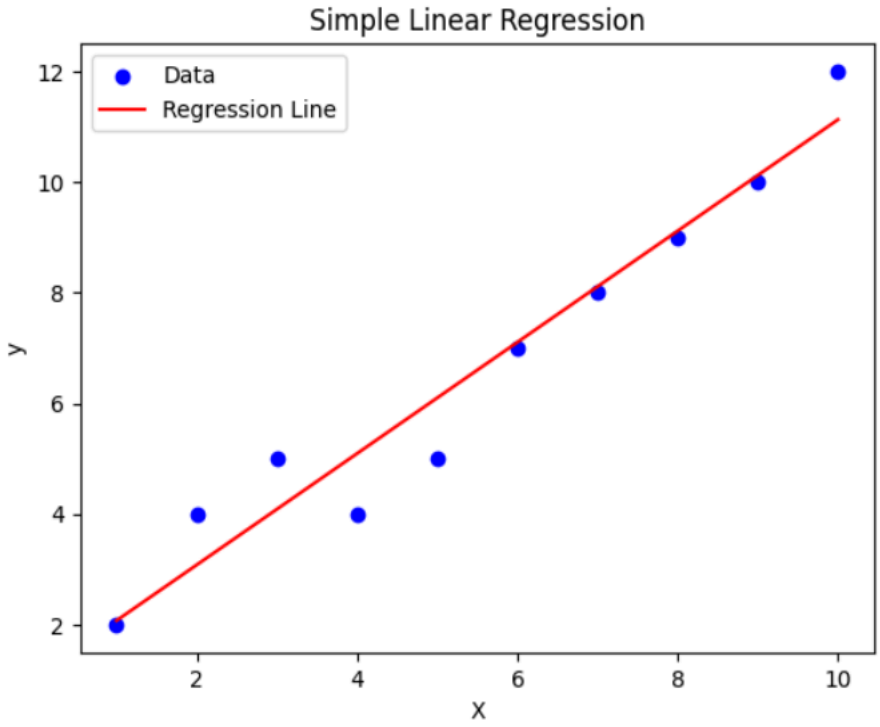
Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To obtain the best fit line over single feature scattered datapoints using Linear Regression	
Experiment No: 02	Date: 10/2/2026	Enrollment No: 92301733059

```
        self.r2score_ = None
        self.poly_features = None

    def fit(self, X, y):
        self.poly_features = PolynomialFeatures(degree=self.degree,
include_bias=True)
        X_poly = self.poly_features.fit_transform(X)
        # Normal equation
        self.coefficients_ =
np.linalg.inv(X_poly.T.dot(X_poly)).dot(X_poly.T).dot(y)
        self.intercept_ = self.coefficients_[0]
        # Predictions and R²
        y_pred = X_poly.dot(self.coefficients_)
        self.r2score_ = 1 - (np.sum((y - y_pred)**2) / np.sum((y -
np.mean(y))**2))
        self.y_pred_ = y_pred

    def predict(self, X):
        X_poly = self.poly_features.transform(X)
        return X_poly.dot(self.coefficients_)

X_poly = np.random.rand(50,1)*6 - 3
y_poly = 0.5*X_poly**2 + X_poly + 2 + np.random.randn(50,1)*0.5
y_poly = y_poly.flatten()
pr = PolynomialRegression(degree=2)
pr.fit(X_poly, y_poly)
print(f"Polynomial LR Coefficients: {pr.coefficients_}")
print(f"R² Score: {pr.r2score_:.2f}")

# Plot
plt.scatter(X_poly, y_poly, color='blue', label='Data')
plt.plot(np.sort(X_poly, axis=0), pr.y_pred_[np.argsort(X_poly, axis=0)],
color='red', label='Polynomial Fit')
plt.title("Polynomial Regression")
plt.xlabel("X")
plt.ylabel("y")
plt.legend()
plt.show()
```

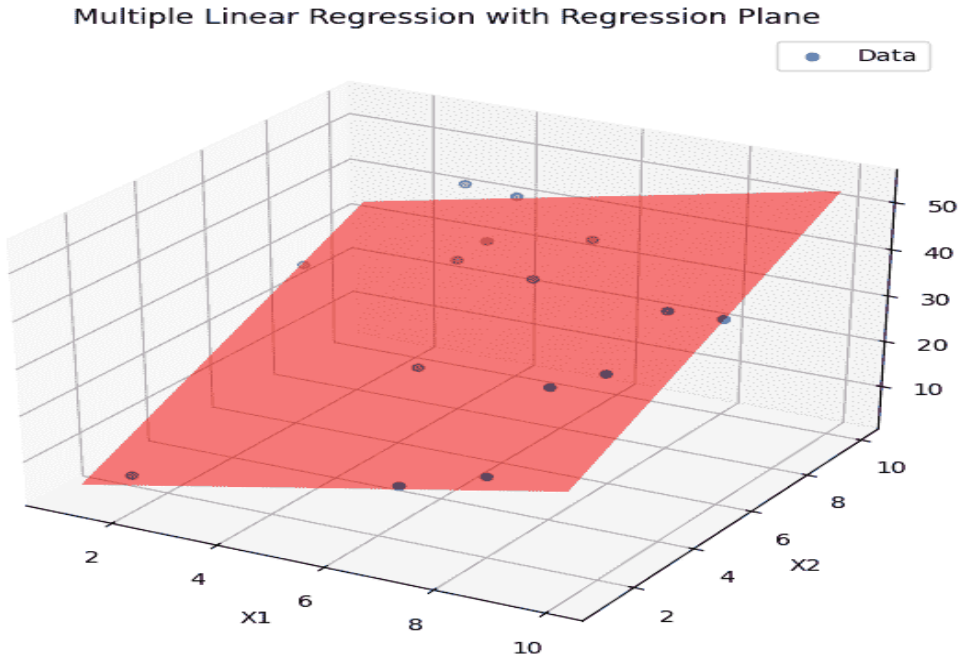
Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To obtain the best fit line over single feature scattered datapoints using Linear Regression**	
Experiment No: 02	**Date: 10/2/2026**	**Enrollment No: 92301733059**

Results:

```
Simple LR Coefficients: [1.06666667 1.00606061]
R² Score: 0.94
```

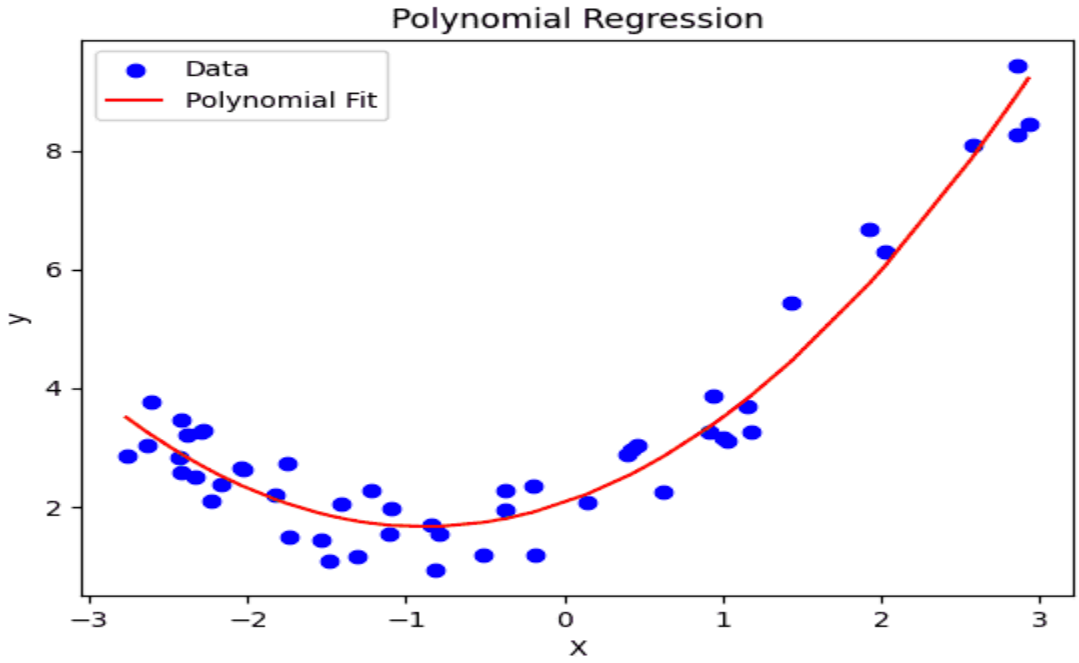



```
Multiple LR Coefficients: [-1.45333931  2.15806389  3.32011421]
R² Score: 0.95
```



Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To obtain the best fit line over single feature scattered datapoints using Linear Regression**	
Experiment No: 02	**Date: 10/2/2026**	**Enrollment No: 92301733059**

```
Polynomial LR Coefficients: [2.06625067 0.91748431 0.51974149]
R² Score: 0.94
```



Observation and Result Analysis:

a. **Nature of the dataset**

1. Attributes: It typically consists of continuous numerical values. In the example, Salinity is used as the independent variable (X) and Temperature as the dependent variable (y).

2. Relationship: Initial observation through a scatter plot often shows a non-linear or complex relationship when the entire dataset is used.

3. Data Integrity: The raw dataset often contains NaN (missing values), which require cleaning (using fillna or dropna) before the training process can begin.

b. **During Training Process**

1. Data Splitting: The dataset is divided into a Training Set (e.g., 75-80%) and a Testing Set (20-25%). This ensures the model is trained on one portion and validated on another.

2. Fitting the Model: Using the LinearRegression().fit() method from the sklearn library, the algorithm calculates the coefficients (slope) and intercept that define the "best-fit" line.

c. **After the training Process**

1. Accuracy Score: A low accuracy score (e.g., 0.12 or 12%) is often observed when using the full dataset. This suggests that a simple linear model does not fit the existing data well because the relationship is not globally linear.

2. Optimization: To improve results, the tutorial observes that a smaller subset of data (e.g., the first 500 rows) might follow a linear pattern more closely. Training on this subset significantly increases the accuracy score.

3. Visual Analysis: Plotting the test data against the predicted line (the regression line) allows for a visual confirmation of how well the model generalizes. If the points are far from the line, the model has high error.

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To obtain the best fit line over single feature scattered datapoints using Linear Regression**	
Experiment No: 02	**Date: 10/2/2026**	**Enrollment No: 92301733059**

Post Lab Exercise:

a. What are the major assumptions considered in linear regression

For a linear regression model to provide reliable and unbiased results, the following key assumptions must be met:

- Li**nearity:** The relationship between the independent variables (X) and the dependent variable (Y) must be linear.
- **Independence:** The observations in the dataset must be independent of each other (no autocorrelation).
- **Homoscedasticity:** The variance of residual errors should be constant across all levels of the independent variables. If the variance "fans out," it is called heteroscedasticity.
- **Normality of Residuals:** The error terms (residuals) should follow a normal distribution, which is important for hypothesis testing and calculating confidence intervals.
- **No Multicollinearity:** The independent variables should not be highly correlated with each other. High correlation makes it difficult to determine the individual effect of each variable.

b. Why MSE is used instead of MAE for calculating the loss function

While both Mean Squared Error (MSE) and Mean Absolute Error (MAE) measure prediction error, MSE is often preferred as a loss function for several reasons:

- **Mathematical Differentiability:** MSE (e^2) is a continuous and differentiable function everywhere. MAE (|e|) has a "V" shape with a sharp corner at zero, where the derivative is undefined. This makes MSE much easier to use with optimization algorithms like Gradient Descent.
- **Sensitivity to Outliers:** MSE squares the error, meaning larger errors are penalized significantly more than smaller ones. This forces the model to prioritize reducing large deviations.
- **Statistical Properties:** Under the assumption that errors are normally distributed, the MSE corresponds to the Maximum Likelihood Estimation, making it statistically efficient for finding the "best" line.

c. How can the behaviour of outliers be understood while dealing with the unseen dataset

When dealing with an unseen (test) dataset, outliers can drastically shift the model's predictions. You can understand their behavior through:

- **Residual Analysis:** Plotting the residuals of the unseen data. If a few points have significantly higher residuals than the rest, they are likely outliers that the model failed to generalize.
- **Influence Statistics:** Tools like Cook's Distance help determine how much the model's coefficients would change if a specific data point were removed.
- **Leverage:** High-leverage points are those with extreme X values. Even if they follow the trend, they can "pull" the regression line toward them, potentially masking errors in other parts of the data.

d. Derive the Normal Equation for the Linear Regression.

The Normal Equation is an analytical way to find the coefficients (θ) that minimize the Cost Function $J(\theta)$ without using iterations.1. The Cost Function (Matrix Form):The goal is to minimize the Sum of Squared Errors:

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To obtain the best fit line over single feature scattered datapoints using Linear Regression**	
Experiment No: 02	**Date: 10/2/2026**	**Enrollment No: 92301733059**

1. The Cost Function (Matrix Form):

The goal is to minimize the Sum of Squared Errors:

$$J(\theta) = \|X\theta - y\|^2 = (X\theta - y)^T(X\theta - y)$$

2. Expand the Expression:

$$J(\theta) = ((X\theta)^T - y^T)(X\theta - y)$$

$$J(\theta) = \theta^T X^T X\theta - \theta^T X^T y - y^T X\theta + y^T y$$

Since $\theta^T X^T y$ is a scalar, it is equal to its transpose $(y^T X\theta)$. Thus:

$$J(\theta) = \theta^T X^T X\theta - 2(X^T y)^T\theta + y^T y$$

3. Take the Derivative with respect to θ:

To find the minimum, set the partial derivative to zero:

$$\frac{\partial J(\theta)}{\partial \theta} = 2X^T X\theta - 2X^T y = 0$$

4. Solve for θ:

Divide by 2 and rearrange:

$$X^T X\theta = X^T y$$

$$\theta = (X^T X)^{-1} X^T y$$

This final formula, $\theta = (X^T X)^{-1} X^T y$, is the **Normal Equation**. It provides the optimal parameters in a single calculation, provided that $(X^T X)$ is invertible.